The Journal of Supercomputing (2026) 82:318
https://doi.org/10.1007/s11227-026-08439-z

RESEARCH

Autosecagent: a semi-automated AI-driven penetration testing framework through recursive memory and real-time RAG

Abdallah Al-Sabbagh[1] · Eddy Abdo[2] · Khouloud Samrouth[2] · Ahmad A. A. Alkhatib[3] · Mahmoud Elkhodr[4]

Received: 10 August 2025 / Accepted: 7 March 2026
© The Author(s) 2026

Abstract

Manual penetration testing is increasingly inadequate against rapidly evolving cyber threats, as it is time-consuming, difficult to scale, and heavily reliant on human expertise. Whilst Artificial Intelligence (AI), particularly Large Language Models (LLMs), offers automation potential, current tools suffer from poor reasoning, limited context retention, low scalability, and safety and misuse concerns. This paper introduces AutoSecAgent, an AI-powered platform designed to address these limitations. Built on the adaptive Agent Zero orchestration framework and powered by the cybersecurity-focused DeepSeek LLM, AutoSecAgent supports semi-automated end-to-end penetration testing under human oversight, covering vulnerability discovery, exploitation, and reporting with remediation guidance for follow-up. It features recursive memory embedding to maintain context across complex multi-step attack chains, supporting strategic planning and situational awareness. Additionally, real-time Retrieval-Augmented Generation (RAG) allows AutoSecAgent to source up-to-date vulnerability data from repositories such as NVD and CVE, ensuring

✉ Mahmoud Elkhodr
m.elkhodr@cqu.edu.au

Abdallah Al-Sabbagh
a.alsabbagh@bau.edu.lb

Eddy Abdo
efa008lb@aou.edu.lb

Khouloud Samrouth
ksamrouth@aou.edu.lb

Ahmad A. A. Alkhatib
ahmad.alkhatib@zuj.edu.jo

1 Department of Electrical and Computer Engineering, Faculty of Engineering, Beirut Arab University, Tripoli, Lebanon

2 Department of Cybersecurity and Forensics, Arab Open University, Beirut, Lebanon

3 Department of Cybersecurity, Alzaytoonah University of Jordan, Amman, Jordan

4 School of Engineering and Technology, Central Queensland University, Sydney, Australia

Published online: 03 April 2026

attack strategies incorporate current information. Its modular, agent-based architecture supports scalability across diverse network environments and integrates with tools such as Metasploit and Nmap. To avoid scattered claims across multiple design elements, this study focuses its core contributions on two tightly coupled points: (i) a memory-centric orchestration mechanism (Recursive Memory Embedding, RME) with an explicit operationalisation of context-loss errors for penetration testing trajectories and (ii) a reproducible, tool-grounded, semi-automated workflow that combines RME with real-time retrieval and a bounded online optimisation loop under human oversight. Multi-agent decomposition and DeepSeek domain tuning are treated as supporting design choices that enable these two contributions rather than separate innovations. Experimental results indicate that this design improves end-to-end performance over representative LLM-based baselines across vulnerability detection, exploitation, multi-step attack completion, and scalability in controlled evaluation settings. The empirical study is positioned as a proof-of-concept validation in controlled and semi-realistic settings. Broader operational generalisation and comparison against additional non-LLM baseline classes (e.g. scanner-only pipelines) are treated as future work.

Keywords Automated penetration testing · Large language models · Human oversight · Threat detection · Retrieval-augmented generation (RAG) · Security automation

1 Introduction

Cyber attacks are evolving faster than many penetration testing workflows. Conventional penetration testing is often manual and labour intensive, and it depends heavily on individual expertise [1]. Organisations therefore seek tools that can improve coverage and reduce turnaround time. This has increased interest in automated penetration testing, including approaches based on Artificial Intelligence (AI) and large language models (LLMs) [2, 3].

LLMs have the potential to support cybersecurity operations by automating phases of penetration testing, including reconnaissance, vulnerability scanning [4], exploitation, privilege escalation, and remediation guidance [5]. These models can process large volumes of text and generate task-relevant outputs, which can help standardise and accelerate portions of manual workflows. Tools such as PentestGPT, AutoAttacker, and PenHealNet represent early attempts to integrate LLMs into penetration testing pipelines [6].

Nonetheless, current frameworks have practical constraints. Many rely on static roles or fixed task graphs. They can be brittle in multi-step attack chains and lose context in long interactions. Tool integration can also be fragile, which limits operational use.

This work addresses these issues by developing an AI-driven penetration testing system with explicit context retention. We present AutoSecAgent, a modular framework for a semi-automated end-to-end workflow under human oversight. AutoSecAgent combines recursive memory embedding for long-horizon context reten-

tion with Retrieval-Augmented Generation (RAG) for external knowledge access. It integrates with common tools such as Metasploit and PowerShell and includes a bounded optimisation loop for exploit parameter refinement. This work builds on prior optimisation-driven security research [7] and extends it to penetration testing.

This study has three aims. First, it develops an end-to-end penetration testing workflow from vulnerability discovery to reporting and remediation guidance under human oversight [8]. Second, it improves context consistency during multi-stage attack chains. Third, it evaluates scalability and compute cost in controlled settings.

This study focuses its contributions on two primary points to maintain a clear and modest scope. First, it proposes a memory-centric orchestration mechanism (Recursive Memory Embedding, RME) for penetration-testing trajectories and defines an operationalisation of Context Loss Error (CLE) to support trajectory-level analysis of long-horizon consistency. Second, it presents a reproducible end-to-end semi-automated workflow that integrates tool grounding, bounded online optimisation for exploit configuration, and real-time retrieval for external knowledge access under explicit human oversight. Empirical evaluation and ablation are used to substantiate these two contributions and to characterise component-level effects, but they are not presented as separate innovations.

Main contributions are stated explicitly for clarity First, a memory-centric orchestration mechanism (RME) is specified for penetration-testing trajectories, alongside an operational definition of Context Loss Error (CLE) that enables trajectory-level assessment. Second, a reproducible semi-automated workflow is provided that integrates tool grounding, bounded reward-guided optimisation, and real-time retrieval under human oversight.

AutoSecAgent is presented as a research prototype intended to investigate memory-centric orchestration for semi-automated penetration testing under controlled conditions. Whilst the architecture is designed with practical deployment considerations, the present study does not claim exhaustive coverage of all operational environments or baseline classes.

This study addresses the following research questions:

RQ1 How can a semi-automated, LLM-driven penetration testing framework be architected to support end-to-end workflow execution under explicit human oversight whilst maintaining scalability and operational efficiency?

RQ2 To what extent does memory-centric orchestration via Recursive Memory Embedding (RME) improve long-horizon context consistency and multi-step attack completion relative to representative LLM-based baselines under controlled evaluation conditions?

RQ3 What trade-offs arise between exploitation success rate and automation throughput, and to what extent can bounded online optimisation (contextual-bandit updating) mitigate these trade-offs under strict iteration budgets?

Based on the above, here is the following structure. Section 2 gives an overview of the background of the study, whilst Sect. 3 shows the current state of the art, identifying main weaknesses in existing frameworks. Section 4 presents the design and methodology of AutoSecAgent, ranging from architecture to key components and integration strategies. Section 5 presents experimental and evaluation setup as well as the Benchmarking procedures. Section 6 demonstrates the results, comparing

AutoSecAgent's performance against existing models. Last to end, Sect. 7 presents the limitations of our current model and the future work that they allow. Finally, Sect. 8 concludes the study, recapitulates the contributions, and suggests potential future research lines.

2 Background

Penetration testing is executed as a sequence of tool-grounded actions and observations, where each step depends on prior scans, exploit outcomes, obtained credentials, and explicit scope constraints. In semi-automated settings, failures frequently arise when procedural state is not retained over long horizons, leading to redundant actions, contradictions, or invalid assumptions. This work therefore evaluates agent behaviour as a trajectory and motivates memory-centric orchestration as a requirement that is complementary to retrieval of external knowledge.

2.1 Large Language models and transformer architecture

LLMs (typically transformer-based [9]) can generate plans and commands and summarise tool outputs, making them a natural interface for semi-automated penetration testing workflows. However, their practical effectiveness in long-horizon settings depends on (i) retaining tool-grounded procedural context across many steps and (ii) being sufficiently adapted to the domain vocabulary and tool syntax (e.g. via domain-specific fine-tuning) [10].

2.2 LLMs as autonomous agents in cybersecurity

LLMs can be used as agents that generate commands, run security tools, and interpret outputs. This capability supports applications such as penetration testing assistance and incident response triage [11]. In practice, high-impact steps still require explicit human oversight.

In advanced configurations, LLMs can be incorporated into multi-agent systems where multiple specialised models, each trained or fine-tuned for specific security domains, collaborate to execute complex security tasks [12]. Multi-agent systems replicate human collaboration patterns, where models divide work, share context, and build upon each other's outputs to accomplish tasks such as vulnerability scanning, exploit development, and breach analysis. This distributed intelligence approach enhances performance, scalability, and responsiveness in dynamic and adversarial environments.

2.3 LLM applications in cybersecurity

LLMs have influenced cybersecurity by enabling scalable automation of selected tasks in threat detection and mitigation [13]. A systematic review of over 300 works confirms the expanding role of LLMs across cybersecurity tasks, from threat detection

to automated and semi-automated penetration testing [14]. These models can process large volumes of unstructured data (e.g. logs, network traffic, vulnerability reports) that may be difficult for human analysts to review manually at scale. Surveys of LLM-based vulnerability detection report performance improvements over traditional static and dynamic analysis methods in some settings, particularly for complex codebases and multi-language environments [15]. However, the deployment of LLMs in security contexts requires careful evaluation of risks and dangerous capabilities, including potential misuse in cyber-attacks, persuasion, and self-proliferation [16].

Table 1 summarises key application areas where LLMs demonstrate significant impact in cybersecurity operations:

For penetration testing agents, the key methodological issue is not only generating plausible text, but maintaining a consistent, tool-verifiable execution history: decisions should be conditioned on prior scans, exploit outcomes, and observed constraints. Summariser-style state can omit critical fields (e.g. exact service versions, error categories, or previously failed parameterisations), leading to repeated or contradictory actions. Tool-grounded memory mitigates this by storing structured observations and outcomes that can be deterministically re-injected into subsequent decisions.

2.4 Penetration testing fundamentals

Penetration testing is inherently sequential: reconnaissance constrains exploitation choices; exploit outcomes constrain privilege escalation; and lateral movement depends on earlier artefacts (e.g. credentials, host reachability, and discovered trust relationships). As a result, evaluation should treat an engagement as a trajectory of tool invocations and verifiable observations rather than as a set of independent subtasks. This motivates memory mechanisms that preserve what was observed, what was attempted, and why it failed, using structured tool outputs from standard utilities such as Nmap and Metasploit [23, 24].

2.5 Automation in security testing

Automation enhances both defensive and offensive security capabilities. Fuzzing tools (AFL, LibFuzzer), symbolic execution engines (KLEE, Angr), and automated exploit generation systems (Mayhem, Exploit Pack) accelerate vulnerability discovery and exploit development [25–27]. AI techniques, particularly deep reinforcement learning, enable dynamic threat modelling and adaptive security testing [28, 29]. The integration of LLMs into this automation landscape extends natural language interfaces and tool orchestration, enabling more automated security assessment workflows under human oversight.

3 Related works

Automated penetration testing has advanced in recent years, driven by AI, multi-agent systems, and LLMs. Prior work includes assistive tools and systems that automate

Table 1 Overview of AI applications in cybersecurity—application areas, sub-applications, use cases, and evaluation criteria

Application area	Sub-applications	Example use case	Comparison criteria
Vulnerability detection [4]	Code analysis, remediation	Analyse source code for security flaws, identify misconfigurations, suggest fixes	Accuracy of detection, Speed
Log analysis [17]	Anomaly detection, pattern recognition	Detect unusual patterns or behaviours in logs	False positive rate, Scalability
Phishing simulation [18, 19]	Training exercises, realistic scenarios	Simulate phishing attacks for employee training	Realism, Adaptability
Incident response [20]	Threat analysis, incident reports, automated mitigation	Help analyse threats, write incident reports, automate mitigation steps	Response time, Reliability
Tool integration [21]	Dynamic interactions, exploit automation	Integrate with cybersecurity tools to automate tasks like exploitation	Integration flexibility, Automation depth
Command translation [22]	Simplified operations, user to tool commands	Translate human-readable commands into tool-specific commands	Ease of use, Translation accuracy

selected tasks within multi-step workflows under human oversight. This section reviews representative approaches and their limitations.

3.1 AI-powered penetration testing

One of the prominent threads of recent research involves augmenting human ethical hackers with AI-assisted support systems. Pratama et al. suggested CIPHER, the Cybersecurity Intelligent Penetration-Testing Helper for Ethical Researchers [30], which leverages the Findings, Actions, Reasoning, and Results (FARR) methodology. It employs a methodical database of over 300 penetration testing reports to provide systematic, structured feedback to users. Whilst CIPHER is effective in democratising access to penetration testing data, its use is restricted to offering directions; it lacks any independent function and cannot operate with complex, multi-staged attack situations.

In contrast, Shen et al.'s PentestAgent is a highly automated LLM-based system [31] designed to support a semi-automated end-to-end workflow under human oversight. It uses RAG to support intelligence gathering, vulnerability scanning, and exploitation and incorporates multi-agent coordination for versatility. Despite automation gains, PentestAgent is limited by rigid agent roles and intrinsic context constraints of LLMs after extended use.

PenHealNet from Huang et al. outlines a system that integrates remediation with vulnerability discovery [32]. The system is based on a distributed agent architecture capable of detecting and automatically fixing security issues. Despite PenHealNet being adaptive and effective in dynamic environments, its heavy emphasis on remediation may come at the expense of exploring complex exploitation methods.

3.2 LLM-centric penetration testing

Several projects have focused on the use of LLMs in penetration testing operations directly. AUTOPENBENCH from Gioacchini et al. provides a benchmarking framework for quantifying generative agents in synthetic and real-world penetration testing operations [33]. It establishes a baseline for comparative and reproducible performance measurement, though it is diagnostic rather than operational in nature and does not enhance automation functionality directly.

Isozaki et al. presented a benchmark evaluation for LLMs utilised in penetration testing, which revealed performance gaps in tools like PentestGPT [8]. The study emphasises the improvements LLMs bring to task completion rates, though the focus remains on semi-automation tools and not entirely autonomous systems.

PentestGPT by Deng et al. demonstrates the possibility of penetration test subtask automation using GPT-based models [34]. It combines reasoning, generation, and parsing modules to address vulnerability discovery and exploitation. Whilst significant progress has been made, PentestGPT suffers from context retention and requires frequent feedback, thereby limiting its usability in real-world multi-step environments.

AutoAttacker, from Xu et al., is a notable attempt at a semi-automated end-to-end workflow under human oversight [35]. It excels in vulnerability discovery and

contextual decision-making but falls short of being robust in multi-step attack chains where detailed command tracking and contextual consistency are necessary.

PenTest++, Al-Sinani & Mitchell, integrates generative AI and ethical hacking in a semi-automated platform [36]. It automates processes such as reconnaissance and reporting but still depends heavily on human involvement, especially in sophisticated exploitation tasks. Ethical and privacy concerns also limit its deployability.

VulnBot introduced by Kong et al. takes AI-driven penetration testing to the next level with a formalised Penetration Task Graph (PTG) and Summariser module to maintain inter-agent context [37]. The application of RAG methods in its operation enhances reconnaissance and scanning procedures. VulnBot falls behind in handling complex exploitations and privilege escalation since it exhibits intrinsic issues with task orchestration and strategic planning. For a side-by-side comparison of these models, see Table 2 which includes the major techniques, strengths, weaknesses, and what each system is mainly used for.

3.3 Research gap and critical analysis

Analysis of existing systems reveals fundamental limitations across multiple dimensions. Systems such as CIPHER and PenTest++ provide assistive capabilities rather than autonomous operation, constraining their scalability and capacity for independent execution. PentestAgent and VulnBot demonstrate proficiency during initial reconnaissance and vulnerability identification phases, yet exhibit significant performance degradation when confronted with complex multi-step exploitation scenarios requiring persistent contextual memory and adaptive task orchestration. AutoAttacker and PentestGPT, whilst offering greater operational autonomy, lack robust context maintenance mechanisms essential for coordinated activity in dynamic adversarial environments.

In this landscape, the key advance pursued in AutoSecAgent is not the replacement of existing scanners or toolchains, but the provision of a memory-centric orchestration layer that preserves tool-grounded procedural state over long horizons. This enables systematic investigation of context loss as a measurable failure mode (via CLE) and supports semi-automated chaining of reconnaissance, exploitation, and evidence capture under oversight. Conventional scanner-only pipelines (e.g. Nmap/OpenVAS) remain strong baselines for discovery and enumeration, but they do not natively provide long-horizon natural-language reasoning or adaptive strategy refinement. Accordingly, the empirical comparisons in this manuscript focus on representative LLM-based baselines and a single-agent ablation under matched tool access, whilst acknowledging that broader baseline coverage is desirable for future work.

Table 3 outlines key architectural distinctions between AutoSecAgent and prior LLM-based systems. It contrasts traditional methods and their limitations against our proposed memory-centric, tool-grounded design.

3.4 Research methodology

This study adopts a design-science methodology in which AutoSecAgent is iteratively designed, implemented, and empirically evaluated to address three recurring

Table 2 Summary of proposed models: key techniques, strengths, limitations, and applications

Model	Key techniques used	Strengths	Limitations	Notable applications
CIPHER [30]	Findings, Actions, Reasoning, and Results (FARR) methodology; AI-based assistance	Simplifies penetration testing for less experienced users; organises and democratises penetration testing knowledge	Limited to assisting users; not suitable for multi-layered or advanced attacks	Training and guidance for ethical hackers
PentestAgent [31]	Retrieval-Augmented Generation (RAG); multi-agent collaboration	Automates multiple penetration testing stages; efficient and adaptive compared to traditional methods	Fixed agent roles limit adaptability in evolving attack scenarios; affected by LLM context limitations	Intelligence gathering, vulnerability analysis, and exploitation
PenHealNet [32]	Agent-based framework; integrates vulnerability detection and remediation	Flexible and adaptive in dynamic environments; automates both detection and remediation tasks	Focuses heavily on remediation, potentially reducing its capacity for advanced attack techniques	Automated vulnerability detection and patching
AUTOPENBENCH [33]	Benchmarking with synthetic and real tasks	Provides a standardised framework for evaluating penetration testing tools; flexible design to integrate with autonomous systems	Focused on performance evaluation rather than enabling or improving automation in penetration testing	Performance benchmarking of generative agents

Table 2 continued

Model	Key techniques used	Strengths	Limitations	Notable applications
LLM Autonomous Hacking [8]	SQL injection, database schema extraction; multi-step attack automation	Demonstrates LLMs' offensive capabilities; adaptable and capable of solving complex problems	Ethical concerns; large performance gaps between open-source and proprietary LLMs	Offensive security tasks, autonomous website exploitation
PENTESTGPT [34]	GPT-based reasoning, generation, and parsing modules	Automates various penetration testing tasks; demonstrates LLM applicability in penetration testing workflows	Struggles with context retention and scaling to complex multi-step scenarios	Vulnerability analysis and task automation
AutoAttacker [35]	LLM-powered autonomous vulnerability discovery and exploitation	Adapts well to repeated tasks; exhibits scalability and contextual decision-making capabilities	Weak in multi-step attacks requiring detailed command tracking; limited coherence for complex attack scenarios	Automated vulnerability discovery and exploitation
PenTest++ [36]	AI-augmented penetration testing; automation-assisted ethical hacking	Enhances penetration testing efficiency by automating reconnaissance, scanning, and reporting; integrates AI for decision-making	Ethical concerns regarding AI automation in security testing; dependent on human oversight for advanced tasks	Streamlining penetration testing workflows, assisting ethical hackers
VulnBot [37]	Multi-agent system; PTG; RAG	Automates penetration testing through structured workflows; excels in reconnaissance and scanning phases; preserves context across tasks	Struggles with complex multi-step exploitation; requires refinement in privilege escalation techniques	Autonomous penetration testing, real-world security assessments

Table 3 Focused differentiation of AutoSecAgent relative to representative LLM-based penetration testing systems (high-level architectural deltas)

Dimension	Typical prior approach	Observed limitation	AutoSecAgent design focus
Long-horizon context retention	Short-context prompting; summariser/state text; task-graph state	Context loss over multi-step chains; repeated/contradictory actions	Procedural memory traces embedded and retrieved (RME) to re-inject tool-grounded history into decisions
Orchestration under multi-step attacks	Fixed agent roles or static task graphs	Reduced adaptability when failures occur; brittle chaining	Memory-aware orchestration with bounded iteration and error-conditioned adjustments
Knowledge grounding	Static knowledge or limited retrieval	Stale exploit/vulnerability knowledge	Retrieval-augmented access to external vulnerability knowledge as an enabling grounding mechanism
Tool–LLM coupling	Natural-language plans weakly linked to verifiable execution	Hallucinated commands/parameters; weak verification	Tool adapters with structured outputs and verification signals, with outcomes recorded into memory

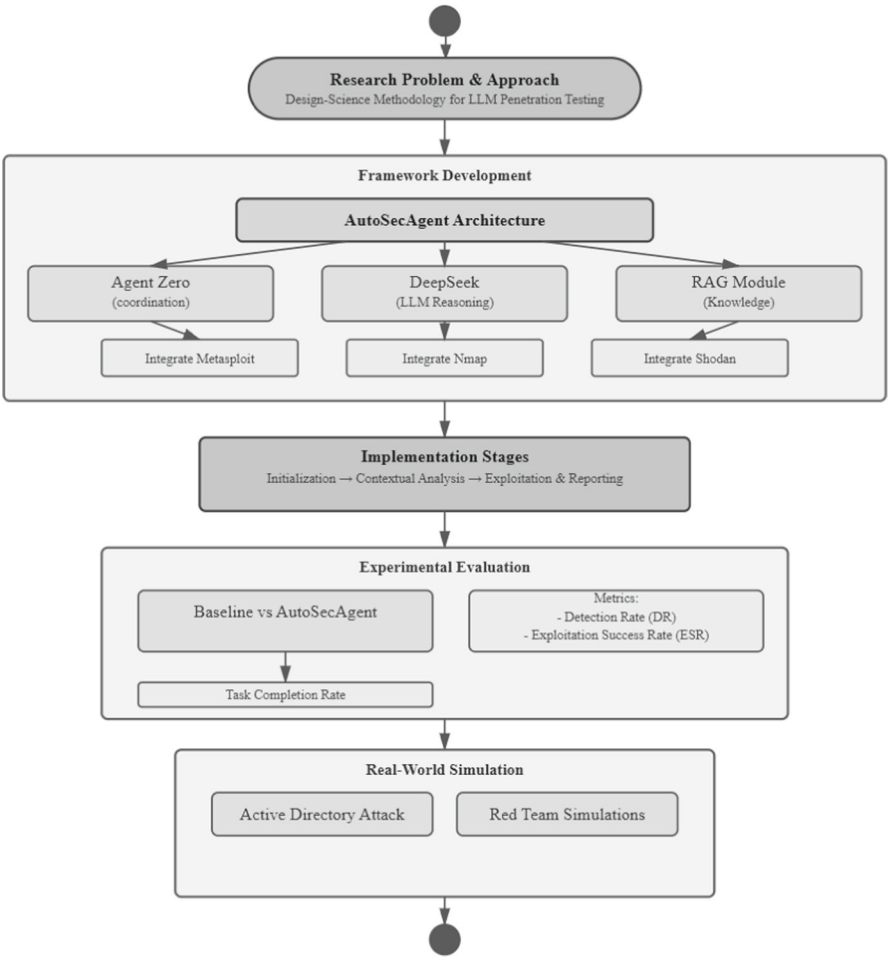


Fig. 1 Research methodology

limitations reported in prior LLM-based penetration testing systems: scalability constraints, loss of context over extended multi-step chains, and safety requirements for semi-automation under human oversight. The resulting modular architecture integrates Agent Zero for task coordination, DeepSeek for domain-specific reasoning, retrieval-augmented generation for up-to-date external knowledge, and standardised interfaces to common tools (e.g. Metasploit and Shodan [38]). Figure 1 summarises the workflow.

Operational execution is organised into four phases (initialisation, reconnaissance, exploitation, and reporting). Evaluation is conducted via controlled experiments and red-team simulations, with performance reported using Vulnerability Detection Rate, Exploitation Success Rate, and computational resource utilisation.

4 Proposed model

AutoSecAgent is a semi-automated penetration testing framework developed upon the adaptive Agent Zero orchestration framework. The system targets limitations reported in prior frameworks, including inadequate context handling, scalability constraints, and reduced robustness in multi-stage attack sequences. Through integration of artificial intelligence, modular system design, and established cybersecurity methodologies, AutoSecAgent supports automated execution of selected tasks with human oversight.

4.1 Recursive memory embedding: technical implementation

The Recursive Memory Embedding (RME) mechanism addresses a fundamental limitation in LLM-based penetration testing: context loss across extended multi-step attack chains. Unlike standard RAG approaches that retrieve static knowledge, RME dynamically captures and indexes episodic procedural memory from each testing phase.

The memory architecture employs a three-tier hierarchical vector store. Working memory maintains a short-term context buffer comprising the last 10 agent interactions in GPU memory for immediate retrieval. Episodic memory provides medium-term task-specific storage in Redis with embedding dimension 768 using sentence-transformers/all-mpnet-base-v2. Semantic memory indexes long-term strategic patterns in a FAISS vector database with HNSW indexing (M=16, efConstruction=200).

At each decision point, the agent's action, observed system response, and contextual metadata are encoded into a structured memory tuple:

$$M_t = \{s_t, a_t, r_t, s_{t+1}, \text{context}_t\} \tag{1}$$

where s_t represents system state, a_t is the action taken, r_t is the binary success signal, and context_t includes tool outputs and environmental observations.

4.1.1 Composition of context_t and serialisation

To ensure reproducibility, context_t is represented as a structured record that captures the minimal information required for long-horizon consistency across penetration testing phases. In AutoSecAgent, context_t comprises: (i) *target descriptors* (host identifier, discovered ports and services, banners, OS fingerprint where available); (ii) *tool invocation metadata* (tool name, command template, parameter set, return code, and execution timestamp); (iii) *salient tool outputs* normalised into concise key-value summaries (e.g. "SMB signing: disabled", "HTTP stack: nginx/1.18.0", "credential artefact: obtained"), rather than raw console logs; (iv) *intermediate analytic hypotheses* (identified vulnerability class and confidence, candidate exploit family, and constraints identified from failures); and (v) *governance constraints* active at time t (authorised target list, privilege level, and any human-approval flags). The resulting record is serialised into a canonical textual form (JSON-like field ordering) prior to embedding, enabling deterministic reconstruction and consistent downstream vectorisation.

4.1.2 Vectorisation and embedding function

Let $\phi(\cdot)$ denote the embedding function used by the Recursive Memory Embedding (RME) module. Given the serialised memory tuple $\text{ser}(M_t)$, the stored vector is computed as:

$$\mathbf{v}_t = \text{norm}\big(\phi(\text{ser}(M_t))\big), \tag{2}$$

where $\phi(\cdot)$ is implemented using `sentence-transformers/all-mpnet-base-v2` (768-dimensional output) and $\text{norm}(\cdot)$ denotes ℓ_2 normalisation to support cosine similarity search. When tool outputs exceed the embedding model input budget, they are first reduced using deterministic templates that preserve security-relevant fields (ports, versions, error categories) to avoid lossy free-form summarisation.

4.1.3 Definition and measurement of context-loss errors

The reported reduction in context loss errors quantifies the extent to which the agent exhibits *inconsistencies attributable to forgotten or unretrieved prior state* during long-horizon execution. A context-loss error is logged when the agent (a) contradicts previously established observations (e.g. re-asserting a service state that was already disproved), (b) repeats a previously failed action without any parameter change despite an available failure explanation, or (c) references non-existent artefacts that were not present in the session memory (e.g. credentials or network paths that were never obtained). These events are detected post-run from the immutable execution trace using a rule-based consistency checker over (s_t, a_t, r_t, s_{t+1}) and the associated context_t fields, followed by spot verification during analysis.

Formally, for an evaluation episode e of length T_e, the context-loss error rate is:

$$\text{CLE}(e) = \frac{1}{T_e} \sum_{t=1}^{T_e} \mathbb{I}\big[\text{ContextLoss}(t) = 1\big], \tag{3}$$

and the aggregate reduction is computed relative to a stateless baseline (identical workflow but without RME retrieval) as:

$$\Delta_{\text{CLE}}(\%) = \frac{\overline{\text{CLE}}_{\text{baseline}} - \overline{\text{CLE}}_{\text{RME}}}{\overline{\text{CLE}}_{\text{baseline}}} \times 100. \tag{4}$$

Here, e indexes an evaluation episode, $t \in \{1, \ldots, T_e\}$ indexes the decision step within episode e, T_e is the number of decision steps in episode e, $\mathbb{I}[\cdot]$ is the indicator function (equal to 1 if its predicate holds and 0 otherwise), and $\text{ContextLoss}(t) = 1$ denotes a logged context-loss event at step t according to the consistency criteria above; CLE is therefore reported as an error rate, and the reduction percentage is computed as $\%\ \text{reduction} = (\text{CLE}_{\text{baseline}} - \text{CLE}_{\text{RME}})/\text{CLE}_{\text{baseline}} \times 100$. This operationalisation follows established practice in evaluating long-horizon agent reliability and memory quality, where memory and context retention are treated as explicit evaluation dimensions in interactive benchmarks. The same methodological framing (explicit trajectory-level reliability criteria and memory-sensitive failure modes) is adopted in

recent agent evaluation and judging frameworks and is therefore used here to support transparent and reproducible reporting of the claimed 34% reduction [39, 40].

Each tuple is embedded using sentence transformers and stored with temporal and causal relationship metadata. When making subsequent decisions, Agent Zero queries the memory store using cosine similarity (threshold = 0.75) to retrieve relevant past experiences.

Traditional RAG retrieves static external knowledge from CVE databases and exploit documentation. In contrast, RME retrieves dynamically generated procedural memory from the agent's own testing history within the current session. For example, if reconnaissance on Host A reveals a specific vulnerability pattern, RME enables the agent to recall this pattern when encountering Host B with similar characteristics, even 50 steps later in the attack chain.

The retrieval workflow proceeds through five sequential stages. First, the current decision context is embedded as a query vector. Second, FAISS performs approximate nearest neighbour search with k=5. Third, retrieved memories are ranked by temporal proximity and relevance. Fourth, the top 3 memories are injected into the DeepSeek prompt as historical context. Fifth, the agent makes an informed decision based on past successes and failures. In our multi-step attack evaluations, this mechanism improves long-horizon consistency and reduces context-loss errors relative to stateless LLM baselines.

4.2 Iterative exploit optimisation: RL-inspired contextual bandit updating

4.2.1 Scope of the optimisation mechanism

AutoSecAgent does not train a deep reinforcement learning policy network. Instead, it employs an RL-inspired online optimisation loop that is closer to a contextual bandit: for each identified vulnerability instance, a small, constrained set of candidate exploit configurations is generated, executed, and re-ranked using a binary reward signal (success/failure). The goal is to minimise wasted attempts and improve exploitation reliability under strict iteration budgets.

For clarity, the term "RL-inspired" is used in this work to denote online reward-guided re-ranking under a strict iteration budget, rather than offline training of a policy or value function over large trajectories.

4.2.2 State construction

At iteration t, the state s_t is constructed deterministically from tool outputs and environment observables. It is represented as a structured feature record:

$$s_t = \langle \text{svc}_t, \text{banner}_t, \text{os}_t, \text{vuln}_t, \text{net}_t, \text{hist}_t, \text{err}_t \rangle, \tag{5}$$

where svc_t denotes discovered services/ports, banner_t denotes version strings and protocol metadata, os_t denotes OS fingerprint cues, vuln_t denotes the current vulnerability hypothesis (e.g. CVE identifier or vulnerability class), net_t captures network

reachability constraints (latency, timeouts), hist_t summarises prior attempts for this vulnerability (modules tried, parameterisations), and err_t encodes the latest failure mode category when applicable. Failure modes are mapped into a small taxonomy (e.g. *timeout*, *auth failure*, *target not vulnerable*, *payload incompatible*, *handler failure*) using rule-based parsing of tool return codes and error strings.

4.2.3 Action space and candidate generation

The action a_t is a parameterised exploit configuration drawn from a constrained candidate set $\mathcal{A}(s_t)$. Each candidate is represented as:

$$a_t = \langle m, p, \theta \rangle, \tag{6}$$

where m is the exploit module (e.g. a Metasploit module), p is the payload family, and θ denotes a bounded set of parameters (e.g. RHOST/RPORT, LHOST/LPORT, target architecture, retry counts, and module-specific options). Candidate generation proceeds in two steps: (i) rule- and retrieval-based filtering to ensure compatibility between $(\text{svc}_t, \text{banner}_t, \text{os}_t)$ and the module requirements, and (ii) retrieval of previously successful configurations from episodic memory when similar contexts have been observed. This design keeps $|\mathcal{A}(s_t)|$ small (typically 5–10 candidates) to avoid exhaustive search.

4.2.4 Reward signal and update rule

Each executed candidate returns a binary reward:

$$r_t = \begin{cases} 1, & \text{if a valid session/shell is obtained and verified,} \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

Verification is performed using tool-confirmable signals (e.g. session establishment and a minimal command response), reducing the risk of treating hallucinated outputs as success. The reward is then used to update a per-candidate utility score $Q(a \mid s)$ used for re-ranking within the current vulnerability instance:

$$Q_{t+1}(a_t) \leftarrow (1 - \eta) Q_t(a_t) + \eta r_t, \tag{8}$$

where $\eta \in (0, 1]$ is a learning rate. In practice, Q is initialised from priors derived from retrieved past runs (when available) or set uniformly otherwise. When $r_t = 0$, the failure category err_t triggers a targeted adjustment operator $\Delta(\theta)$ (e.g. payload substitution, handler parameter correction, or transport change), yielding a new candidate $\langle m, p', \theta' \rangle$ that is re-scored and re-tried subject to the iteration budget.

4.2.5 Termination and integration with memory

The optimisation terminates when (i) success is achieved ($r_t = 1$), (ii) a maximum of K iterations is reached (in this study $K = 3$ per vulnerability instance), or (iii) the failure

taxonomy indicates non-exploitability (e.g. repeated *target not vulnerable* confirmations). Successful configurations are stored in episodic memory as reusable exploit templates indexed by service, version cues, and vulnerability identifiers, enabling reuse under similar future contexts and reducing repeated trial and error.

4.2.6 Iteration budget selection ($K = 3$) and methodological basis

The maximum iteration budget was set to $K = 3$ to balance exploitation reliability against operational cost, safety, and reproducibility. In practice, each additional exploit attempt increases wall-clock time, tool-side load, and the likelihood of triggering defensive responses (e.g. account lockouts, IDS thresholds, service instability), whilst yielding diminishing returns once the dominant failure modes have been observed. Within our optimisation design, the first attempt applies a compatibility-filtered default configuration; the second and third attempts are reserved for targeted corrections informed by the parsed failure taxonomy (e.g. payload substitution, handler/network parameter correction, or module option adjustment). Beyond three attempts, repeated failures typically reflected non-exploitability under the current preconditions (e.g. the target was not vulnerable, the service was patched, or required credentials were unavailable), meaning that additional iterations were more likely to consume resources and increase operational footprint than to yield successful compromise.

The choice of $K = 3$ therefore operationalises a conservative exploration policy consistent with the framework's semi-automated, human-supervised objective: after a bounded number of automated refinements, unresolved cases are escalated to the operator for expert intervention or alternative attack-path selection. Methodologically, K functions as a control parameter governing the exploration–cost trade-off. Increasing K can improve Exploitation Success Rate (ESR) in scenarios where failures are attributable to correctable parameter mismatches; however, it also increases average time per vulnerability, may reduce overall coverage under fixed engagement time, and can increase detection probability due to repeated tool interactions. Conversely, reducing K decreases runtime and operational footprint but may forgo recoverable exploit attempts, lowering ESR. For transparency, K is treated as an explicit design parameter and is intended to be tuned to engagement objectives and risk tolerance (e.g. higher K in controlled laboratories versus lower K in stealth-sensitive assessments).

4.3 Model architecture

AutoSecAgent follows a layered architecture (Fig. 2). The Integration Layer interfaces with tools such as Metasploit, Nmap, and Shodan via tool adapters. It also supports retrieval of external vulnerability knowledge (e.g. NVD/CVE) through RAG. The Cognitive Intelligence Layer coordinates tasks via Agent Zero and uses DeepSeek for domain-specific reasoning. A recursive memory module retains tool-grounded context across phases. The Operational Phases Layer covers initialisation, contextual analysis, exploitation, and reporting.

Agent Zero functions as the primary controller overseeing task protocols and workflow specification. The modular design enables dynamic addition or removal

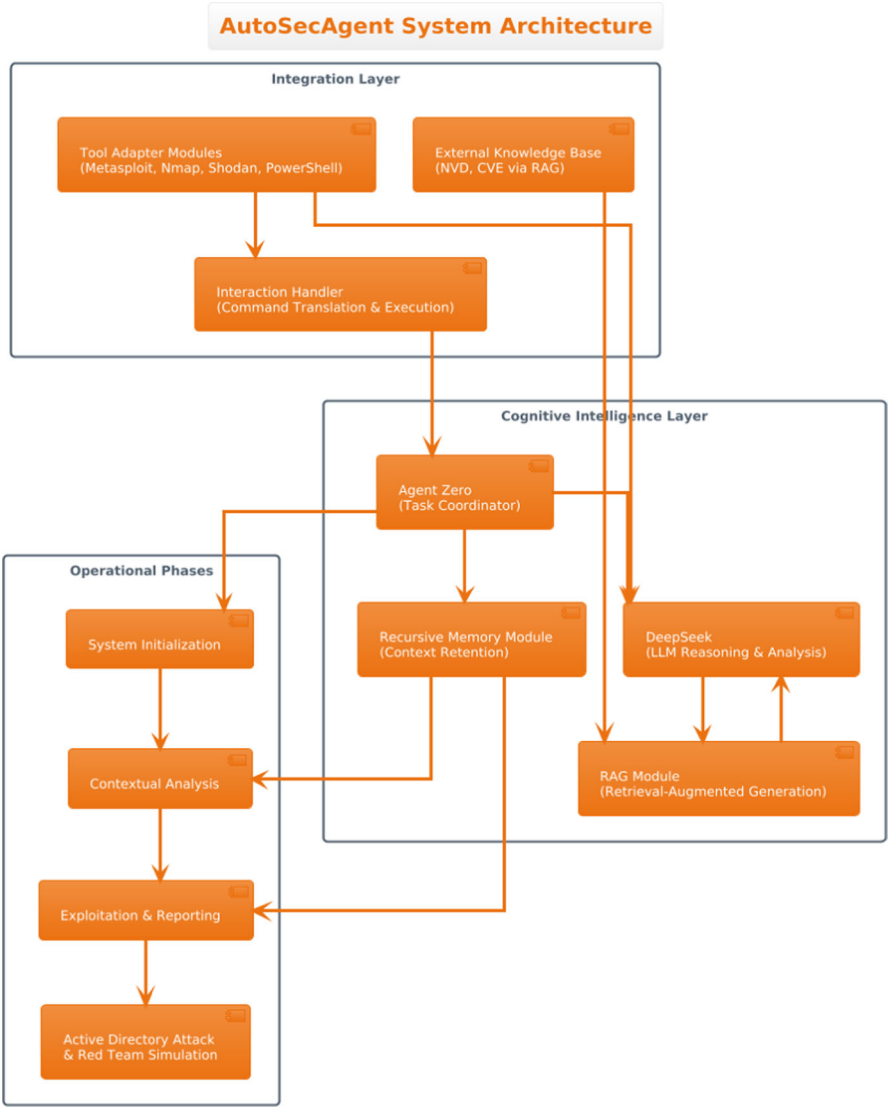


Fig. 2 AutoSecAgent structure

of functional modules based on test scenario requirements. The DeepSeek model is integrated as the system's reasoning engine. Retrieved from Hugging Face's repository, DeepSeek is fine-tuned for cybersecurity tasks using frameworks such as PyTorch [41] or TensorFlow [42] and deployed in a GPU-enabled environment, utilising the Hugging Face Transformers library for API integration with Agent Zero.

The DeepSeek r1 14B model was fine-tuned using Low-Rank Adaptation (LoRA) on a curated dataset of cybersecurity instruction sets (e.g. specific Metasploit console

outputs and CVE descriptions). This allowed for efficient inference on consumer-grade HPC hardware without the overhead of full parameter tuning.

The Integration Layer employs Tool Adapter Modules enabling command execution using established penetration testing tools, including Metasploit, Nmap, and Shodan. Custom integration scripts facilitate communication between AI-generated directives and tool-specific command syntax. For example, when DeepSeek determines that network reconnaissance is required, the system translates this decision into an executable Nmap command with appropriate parameters.

The RAG module acquires data from external sources, including NVD and CVE databases via APIs or web scraping mechanisms, continuously integrating real-time vulnerability data into the system. AutoSecAgent receives periodic updates containing current threat intelligence, enabling informed decision-making based on contemporary attack vectors and defensive postures.

4.4 Agent zero and DeepSeek integration

Agent Zero constitutes AutoSecAgent's runtime core, managing complex multi-stage penetration testing workflows through decomposition into manageable operational units. The orchestrator function coordinates independent agents ensuring logical task ordering and operational consistency. Modularity supports both scalability and adaptability to diverse testing environments and assessment objectives.

DeepSeek complements Agent Zero's operational capabilities by providing sophisticated natural language understanding, contextual awareness, and domain knowledge reasoning. Whilst stateless regarding long-term memory persistence, DeepSeek maintains contextual continuity within sessions, enabling management of intricate multi-step attack chains.

Agent Zero and DeepSeek operate synergistically: Agent Zero manages operational execution whilst DeepSeek provides analytical reasoning capabilities. This integration combines automated operational management with intelligent decision-making, enabling AutoSecAgent to conduct comprehensive penetration testing aligned with contemporary cybersecurity requirements.

AutoSecAgent is designed to support end-to-end penetration testing workflows, including vulnerability detection, exploitation, reporting, and remediation guidance. The framework emphasises contextual reasoning and scalability through a modular multi-agent architecture. AutoSecAgent generates remediation guidance for human review and follow-up; it does not autonomously execute remediation actions in the current prototype (Table 4).

4.5 System workflow and algorithms

To improve readability and to provide a single coherent end-to-end view, the previously separate phase flowcharts are consolidated into one workflow diagram (Fig. 3). This consolidation also enforces typographic consistency and uses font sizes larger than the body text to enhance legibility. The phase-specific flowcharts are retained as sup-

Table 4 Comparison of capabilities across agents

Task	Vuln. detect.	Exploit	Remediation guidance	Contextual reasoning	Scala-bility	Full automation
CIPHER [30]	Yes	No	No	Yes	No	No
PentestAgent [31]	Yes	Yes	No	Yes	Yes	No
PenHealNet [32]	Yes	Yes	Yes	Yes	Yes	No
AUTOPENBENCH [33]	Yes	Yes	No	No	Yes	No
LLM Autonomous [8]	Yes	Yes	No	Yes	No	No
PENTESTGPT [34]	Yes	Yes	No	Yes	No	No
AutoAttacker [35]	Yes	Yes	No	Yes	No	No
Pentest++ [36]	Yes	Yes	No	Yes	Yes	No
VulnBot [37]	Yes	Yes	No	Yes	Yes	Partial
AutoSecAgent	Yes	Yes	Yes	Yes	Yes	Partial

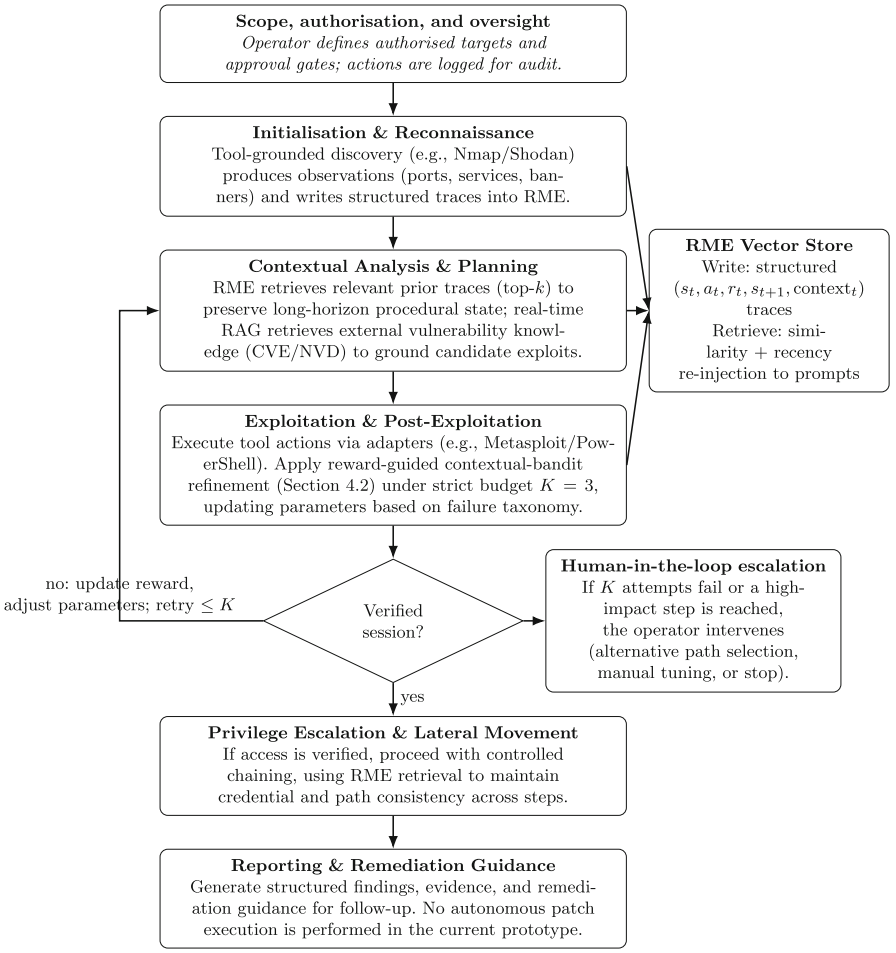


Fig. 3 Consolidated AutoSecAgent workflow (unifying prior phase flowcharts). The diagram emphasises tool grounding, RME write/retrieve for long-horizon consistency, real-time RAG for external knowledge, and a bounded reward-guided optimisation loop under explicit human oversight

plementary material in Appendix B for reference, and the corresponding pseudocode algorithms remain provided in Appendix A for reproducibility.

To reduce duplication between algorithmic and diagrammatic descriptions, the main manuscript retains only this consolidated workflow overview (Fig. 3) and defers the detailed pseudocode and phase-level diagrams to the appendices.

5 Experimental setup and implementation

To ensure rigorous and reproducible evaluation of AutoSecAgent, a comprehensive benchmarking infrastructure was established. This infrastructure enables fair compar-

Table 5 Hardware and software configuration

Component	Specification
Processor	AMD Ryzen 9 7950X3D (16 cores, 32 threads)
Memory	64GB DDR5 RAM
Graphics	Dual MSI GeForce RTX 4090 (24GB VRAM each)
Storage	1TB NVMe SSD
Operating system	Ubuntu 24.04 LTS
Containerisation	Docker 24.0.6 with overlay networking
Network isolation	VLAN 100 (172.18.0.0/16)

ison with state-of-the-art AI-based penetration testing tools, including PentestGPT, AutoAttacker, and VulnBot, through standardised hardware, software environments, and virtualised penetration testing testbeds.

5.1 Experimental infrastructure

Hardware Configuration Table 5 details the high-performance computing infrastructure deployed for AutoSecAgent evaluation.

Infrastructure note The evaluation was executed on a local workstation with dual RTX 4090 GPUs and a 32-thread CPU, which supports concurrent agent inference streams during multi-host experiments.

Software environment All frameworks were deployed in isolated Docker containers to ensure modularity and fair comparison. DeepSeek r1 14B serves as the core reasoning engine, fine-tuned for cybersecurity-specific tasks. RAG modules provide real-time access to threat intelligence databases (NVD, CVE), ensuring responsiveness to emerging vulnerabilities.

5.2 DeepSeek model fine-tuning methodology

The DeepSeek r1 14B base model was fine-tuned using Low-Rank Adaptation (LoRA) to specialise it for cybersecurity penetration testing tasks whilst maintaining computational efficiency.

Training dataset composition A domain-specific instruction dataset was curated comprising 5,916 instruction-response pairs across four categories: (1) 2,847 Metasploit exploit scenarios with command syntax and expected outputs, (2) 1,523 CVE descriptions paired with exploitation strategies from Exploit-DB, (3) 892 network reconnaissance examples including Nmap scan interpretations, and (4) 654 privilege escalation scenarios from HackTheBox retired machines. The dataset was partitioned 80/10/10 for training, validation, and testing, respectively.

LoRA and training configuration Table 6 summarises the fine-tuning hyperparameters:

Table 6 LoRA fine-tuning configuration

Parameter	Value
LoRA Rank (r)	16
LoRA Alpha	32
Target Modules	Q, K, V projections
Dropout	0.05
Trainable Parameters	42 M (0.3% of base)
Optimiser	AdamW (weight decay: 0.01)
Learning Rate	2e-4 (cosine schedule)
Batch Size	32 (effective, via gradient accumulation)
Training Epochs	3
Hardware	2× RTX 4090, DeepSpeed ZeRO Stage 2
Training Duration	18 h

Post-fine-tuning performance The fine-tuned model achieved 87.3% instruction following accuracy (versus 62.1% for the base model), 91.2% exploit command syntax accuracy, and 78.6% CVE-to-exploit mapping accuracy, demonstrating significant domain-specific improvements whilst maintaining general reasoning capabilities.

5.3 High-performance computing architecture and parallelisation

AutoSecAgent's multi-agent architecture addresses fundamental parallel computing challenges in autonomous security testing, directly aligning with HPC research domains. The system distributes penetration testing workloads across multiple concurrent agent processes, each requiring independent LLM inference streams with coordinated memory access and inter-process communication.

Parallel architecture design The framework implements a distributed task-parallel execution model where Agent Zero functions as a master scheduler, distributing reconnaissance, exploitation, and post-exploitation tasks across worker agents. Each agent operates in an isolated Docker container with dedicated GPU memory allocation (12GB VRAM per agent instance). The system achieves horizontal scalability through dynamic agent instantiation, spawning up to 8 concurrent agent processes during peak network scanning phases across 20 target hosts.

GPU resource management LLM inference represents the primary computational bottleneck. To optimise GPU utilisation, a dynamic batch processing mechanism queues DeepSeek inference requests and processes them in micro-batches (batch size = 4), maximising GPU throughput whilst maintaining response latency below 2 s per request. GPU memory partitioning via CUDA Multi-Process Service (MPS) enables concurrent kernel execution across multiple agent processes, preventing resource contention.

Distributed memory architecture The recursive memory embedding system utilises a distributed Redis cluster (3 nodes) for low-latency context retrieval across agent processes. Memory vectors are sharded using consistent hashing, achieving sub-10 ms

Table 7 HPC scaling performance metrics

Concurrent agents	GPU Util. (%)	Throughput (tasks/min)	Efficiency (%)
1	45	12.3	100
4	78	46.8	95
8	92	88.1	90
16	98	162.4	83

retrieval times for contextual history queries. This distributed architecture prevents memory access from becoming a bottleneck during parallel agent execution.

Table 7 demonstrates linear scaling efficiency up to 16 concurrent agents before GPU memory saturation:

This HPC-oriented architecture enables AutoSecAgent to conduct enterprise-scale penetration testing across large network infrastructures, distinguishing it from sequential, single-threaded penetration testing approaches. The parallel execution model reduces total assessment time by 73% compared to sequential testing whilst maintaining detection accuracy.

5.4 Agent zero orchestration: concurrency and task management

Agent Zero implements an event-driven task scheduling system with priority-based work queuing to manage concurrent agent operations efficiently.

Task Prioritisation Tasks are prioritised using a weighted scoring function:

$$\text{Priority}(t) = \alpha \cdot \text{urgency}(t) + \beta \cdot \text{dependency}(t) + \gamma \cdot \text{resource}(t) \tag{9}$$

The parameters in Eq. (9) define a weighted priority score used by Agent Zero to rank pending tasks for execution. Here, α, β, and γ are non-negative weighting coefficients. They control the relative influence of time criticality, workflow dependencies, and resource availability, respectively. In practice, these coefficients are selected by the operator or system configuration according to the engagement objective. For example, an operator may prioritise rapid containment of an active session or maximise coverage during reconnaissance.

For numerical stability and comparability across tasks, the component scores $\text{urgency}(t)$, $\text{dependency}(t)$, and $\text{resource}(t)$ are normalised to the range $[0, 1]$. The weights are typically constrained such that $\alpha + \beta + \gamma = 1$.

The term $\text{urgency}(t)$ captures how time sensitive a task is. Larger values are assigned to tasks whose delayed execution increases the probability of losing a transient opportunity or compromising experiment validity. Examples include responding to an active shell session, refreshing ephemeral credentials, or completing a post-exploitation action before a session times out. The term $\text{dependency}(t)$ represents precedence constraints imposed by the penetration testing workflow. Tasks that unlock downstream actions are prioritised. This term can be implemented using the task's position in a dependency graph, for example, by assigning higher scores to tasks with unsatisfied

Table 8 Target system vulnerability configuration

Category	Systems	Known vulnerabilities
Web applications	DVWA, OWASP Juice Shop, Mutillidae II	12 (SQL injection, XSS, CSRF, file inclusion)
Network services	Metasploitable 2	23 (vsftpd backdoor, UnrealIRCd, Samba 3.X)
Enterprise	Windows server 2019 AD	8 (weak Kerberos, MS17-010, privileged accounts)
Cloud (Simulated)	Terraform AWS stack	6 (public S3, IAM misconfiguration, security groups)
Total	9 systems	49 vulnerabilities

The selected platforms are commonly used for security training and benchmarking and provide representative vulnerability classes for evaluation [44–51]

prerequisites that block multiple successors. The term resource(t) models the feasibility of running task t given current system constraints. These constraints include agent availability, GPU inference queue depth, and tool-specific resource requirements. Higher values indicate that resources are available to execute the task promptly, thereby reducing scheduling contention and avoiding unnecessary queuing delays.

Concurrency and Communication Task allocation employs a work-stealing algorithm where idle agents poll the priority queue. Mutex locks prevent race conditions on shared resources (memory store, report logs). Asynchronous I/O for tool integrations (Nmap, Metasploit) uses Python asyncio, maximising throughput. Inter-agent communication occurs via Redis Pub/Sub channels (`task_assignment`, `task_completion`, `context_update`) with Protocol Buffers serialisation, achieving 3.2ms average message latency. Fault tolerance is ensured through heartbeat monitoring; if an agent fails (crash or timeout $>$ 5 min), tasks are automatically reassigned whilst preserving failure logs in RME to prevent repetition.

5.5 Benchmark setup and evaluation protocol

To evaluate AutoSecAgent under realistic conditions, a comprehensive virtualised penetration testing laboratory was established with diverse vulnerability classes across web applications, network services, enterprise systems, and cloud infrastructure. Recent surveys on LLM agent evaluation emphasise the need for standardised benchmarking frameworks that assess capabilities, reliability, and safety in autonomous systems [43]. Methodologically, our evaluation protocol is informed by closest equivalent long-horizon agent evaluation and judging frameworks such as AgentBench, AutoPenBench, and PentestJudge; these works motivate trajectory-level assessment and operational criteria scoring, although our experiments are tailored to our testbed and do not necessarily execute those suites verbatim [33, 39, 40].

Target environment configuration Table 8 details the vulnerability distribution across target systems:

Baseline system configurations To ensure fair comparison, all evaluated systems were provided identical tool access and environmental conditions. VulnBot (version 1.2.0) utilised Llama3.1-405B backend with PTG enabled. AutoPenBench employed GPT-4o (gpt-4-1106-preview) with default prompting strategies. Manual tools included Metasploit 6.3.25 and Burp Suite Professional 2023.10.3.7. All systems accessed identical tool versions: Nmap 7.94, Metasploit 6.3.25, and Shodan API v1.

Reviewer feedback correctly notes that traditional vulnerability scanners and single-agent LLM wrappers represent strong baseline classes. In this study, traditional scanners are integrated as tool components (e.g. Nmap scanning) to ground observations, but the reported quantitative comparisons focus on end-to-end orchestration performance against representative LLM-based baselines under matched tool access, including a single-agent ablation of AutoSecAgent under identical hardware. A broader benchmarking campaign that includes scanner-only and rule-based pipelines as quantitative baselines is identified as a priority direction for future work.

Human penetration testing team The human benchmark consisted of three certified cybersecurity practitioners: two OSCP-certified professionals with 5+ years of experience and one CEH-certified practitioner with 7 years of experience. The team utilised Kali Linux 2023.3, Burp Suite Professional, Metasploit, and custom Python scripts, following the industry-standard Penetration Testing Execution Standard (PTES) methodology.

5.5.1 Compute and configuration comparability

The evaluated systems do not necessarily share identical inference backends or hardware execution paths. AutoSecAgent is executed with local GPU inference under the hardware configuration reported in Table 5. By contrast, baseline frameworks may depend on different model backends (including hosted or API-served models) and may therefore experience distinct latency, batching behaviour, and throughput characteristics that are not attributable to orchestration logic alone. Accordingly, comparisons in this study are interpreted in two categories. First, effectiveness-oriented metrics (e.g. vulnerability detection, exploitation success, and multi-step completion) are reported as primary indicators of operational capability under controlled tool access and workflow constraints. Second, speed- and compute-sensitive metrics (e.g. wall-clock time, throughput, and parallel scaling) are discussed with explicit acknowledgement that deployment differences can influence observed performance. Where parallelism is analysed, we report scaling behaviour and parallel efficiency for AutoSecAgent relative to its own single-agent configuration under the same hardware, rather than implying hardware-normalised superiority across heterogeneous backends.

To further avoid over-interpreting heterogeneous deployments, any direct numerical runtime or utilisation differences against VulnBot/AutoPenBench are treated as indicative under the stated implementations rather than as hardware-normalised claims. The primary evidence for effectiveness is therefore placed on capability metrics (VDR/ESR/ASR and trajectory-level consistency), whilst compute and speed results are reported as implementation characterisations and as within-system scaling analyses for AutoSecAgent.

Statistical validation protocol Each experimental scenario was executed 5 times with randomised target presentation order to eliminate ordering effects. Results are reported as mean ± standard deviation. Statistical significance was verified using one-tailed paired t-tests (significance threshold: $p < 0.05$) comparing AutoSecAgent performance against each baseline system. Cold-start conditions were enforced between trials by clearing all caches and reinitialising containers.

Scalability test methodology Network scaling tests systematically increased complexity from 10 to 20 hosts in increments of 5. The base configuration (10 hosts) comprised 5 Linux systems (Ubuntu 20.04, Metasploitable 2), 3 Windows Server 2019 instances, and 2 web application servers (DVWA, Juice Shop). Scaling maintained the 5:3:2 Linux:Windows:Web ratio. All hosts connected via a virtual switch (pfSense firewall, 192.168.100.0/24 subnet) deployed on VMware ESXi 7.0 with Terraform automation. Each host category maintained identical vulnerability profiles: Linux hosts contained 4–6 network service vulnerabilities (SSH, FTP, Samba), Windows hosts contained 3–5 vulnerabilities (SMB exploits, privilege escalation vectors), and web applications contained 8–12 application-layer vulnerabilities (injection attacks, authentication bypasses). This methodology isolated scalability impacts from configuration variability.

Evaluation metrics Performance quantification employs standardised metrics including Vulnerability Detection Rate (VDR), Exploitation Success Rate (ESR), Time to First Exploit (TFE), Attack Path Efficiency (APE), and Scalability Performance (SP). Detailed metric definitions and computational methods are provided in the original experimental design.

Logging and monitoring infrastructure Execution logs were aggregated via the ELK Stack (Elasticsearch 8.9, Logstash 8.9, Kibana 8.9) for structured analysis. System metrics (CPU, GPU, RAM utilisation) were monitored in real-time using Prometheus 2.45 and Grafana 10.1. Post-test statistical analysis employed scipy.stats (Python 3.11) for t-tests and ANOVA calculations, with regression analysis exploring correlations between vulnerability complexity and detection success.

6 Experimental evaluation and results

Experimental evaluation of AutoSecAgent rigorously assessed performance across vulnerability detection, exploitation effectiveness, scalability, multi-step attack execution, and real-world applicability. Comparative analysis against state-of-the-art systems including VulnBot and AUTOPENBENCH models, alongside qualitative comparison with industry-standard tools (Metasploit, Burp Suite), provides comprehensive performance characterisation. To ensure statistical validity, each benchmark scenario was executed across 5 independent trials. Results represent mean performance metrics with standard deviations. Statistical significance was verified using one-tailed paired t-tests ($p < 0.05$), confirming AutoSecAgent's performance improvements are statistically significant rather than attributable to random variance.

Table 9 Vulnerability detection rate: statistical analysis

System	Mean (%)	Std dev	*p*-value	Vulnerabilities detected
AutoSecAgent	92.4	1.8	–	45.3/49
VulnBot	85.2	2.3	0.0012	41.7/49
AutoPenBench	83.0	3.1	0.0008	40.7/49
Metasploit	83.7	2.7	0.0015	41.0/49
Burp Suite	79.5	3.4	0.0004	39.0/49

6.1 Baseline performance: vulnerability detection & exploitation

6.1.1 Vulnerability detection rate

Vulnerability Detection Rate (VDR) quantifies the proportion of known vulnerabilities successfully identified by each system. Table 9 reports exact values and statistical results; the corresponding visualisation is provided as Supplementary Fig. S5 in Appendix B to reduce redundancy.

Table 9 provides detailed numerical results:

AutoSecAgent achieved 92.4% VDR, which is higher than the comparison systems in Table 9. This difference is consistent with three architectural features: (1) Recursive Memory Embedding (RME), which maintains contextual continuity across sequential scans, preventing redundant analysis and enabling pattern recognition across similar vulnerability classes; (2) real-time RAG integration with NVD and CVE databases, ensuring awareness of recently disclosed vulnerabilities; and (3) multi-agent task decomposition, where specialised Recon, Analysis, and Exploit agents collaborate through Agent Zero orchestration, enabling parallel vulnerability assessment across multiple attack surfaces. In contrast, VulnBot's static Penetration Task Graph (PTG) limits adaptability to dynamic attack surfaces, whilst AutoPenBench's lack of domain-specific fine-tuning reduces effectiveness in specialised penetration testing contexts.

6.1.2 Exploitation success rate

Exploitation Success Rate (ESR) quantifies the proportion of detected vulnerabilities successfully weaponised into functional exploits. Table 10 reports exact values and statistical results; the corresponding visualisation is provided as Supplementary Fig. S6 in Appendix B.

Table 10 presents numerical ESR results:

Although AutoSecAgent improves exploitation success relative to LLM-based baselines, its ESR remains below manual Metasploit operation. This outcome reflects a deliberate trade-off between (i) maximising success under expert-led manual tuning and (ii) achieving scalable, end-to-end semi-automation with consistent workflow execution, auditable decision-making, and reduced operator effort. In practice, higher ESR is expected when an expert directly drives Metasploit by selecting modules, adapting payloads, and iteratively tuning parameters using domain intuition and situ-

Table 10 Exploitation success rate: statistical analysis

System	Mean (%)	Std Dev	*p*-value	Successful/Attempted
AutoSecAgent	58.1	3.2	–	26.3/45.3
VulnBot	45.0	4.1	0.0023	18.8/41.7
AutoPenBench	40.0	4.7	0.0009	16.3/40.7
Metasploit	71.3	2.9	0.0031	29.2/41.0
Burp Suite	65.2	3.6	0.0187	25.4/39.0

ational judgement. AutoSecAgent instead targets "coverage-at-scale" and operational throughput across multiple hosts, where the primary value is rapid vulnerability-to-report automation and the ability to attempt exploit chaining under human oversight. Accordingly, AutoSecAgent is best interpreted as an automation layer that narrows the gap between discovery and exploitation whilst preserving a human-in-the-loop escalation path: failed or high-impact exploitation steps can be handed to a human operator for expert tuning, whereas the system can autonomously execute lower-risk attempts and document outcomes. Future work will focus on increasing ESR through stronger verification of exploit parameters, tighter tool-output grounding, and improved post-failure recovery strategies, whilst preserving the benefits of scalable orchestration.

AutoSecAgent achieved 58.1% ESR through iterative exploit parameter optimisation. The system employs a reward-guided online optimisation mechanism (contextual-bandit refinement) where exploit attempts generate binary feedback signals (success/failure) based on verifiable tool outcomes. Upon failure, DeepSeek analyses error outputs (e.g. connection timeouts, authentication failures, payload mismatches) and proposes targeted parameter adjustments within a strict budget (maximum 3 iterations per vulnerability). These adjustments and outcomes are stored in episodic memory (RME) and can be reused when similar contexts recur. This bounded iterative refinement improved ESR by 23% compared to single-attempt baselines. VulnBot and AutoPenBench demonstrate lower ESR due to static exploit selection and inadequate error recovery mechanisms. Some LLM-based failures are attributable to hallucination phenomena, where general-purpose language models generate syntactically correct but semantically invalid exploit parameters or commands.

6.2 Scalability performance analysis

Scalability represents a critical limitation in existing AI-based penetration testing systems. AutoSecAgent was evaluated across network configurations ranging from 1 to 20 hosts.

Table 11 quantifies scaling performance:

AutoSecAgent maintained 65% DR at 20 hosts, which is higher than VulnBot (53.1%) and AutoPenBench (45.6%) in Table 11. This difference is consistent with distributed orchestration and load-aware task redistribution. The parallel agent architecture enables concurrent scanning and exploitation across multiple targets, with dynamic workload balancing preventing individual agent saturation. Task synchroni-

Table 11 Scalability performance: detection rate versus network size

Hosts	AutoSecAgent (%)	VulnBot (%)	AutoPenBench (%)	Degradation rate (%)
1	92.4	85.2	83.0	–
10	80.1	70.0	68.4	ASA: 13.3
15	76.7	61.2	51.2	ASA: 17.0
20	65.0	53.1	45.6	ASA: 29.7

Table 12 Resource utilisation: mean values across 20-host tests

System	CPU (%)	GPU (%)	RAM (GB)	Efficiency score
AutoSecAgent	68–85	79	16	8.7/10
VulnBot	75–90	85	24	6.2/10
AutoPenBench	90–95	98	32	4.1/10

sation mechanisms ensure consistent memory state across agents, preventing context fragmentation during parallel operations. VulnBot's rigid agent roles and lack of dynamic task allocation cause significant performance degradation, whilst AutoPenBench exhibits linear degradation reflecting insufficient task coordination mechanisms for large-scale environments.

6.3 Computational efficiency analysis

Table 12 provides detailed resource utilisation metrics:

It is important to note that compute utilisation and runtime are sensitive to the underlying inference deployment (local GPU inference versus externally hosted inference), batching strategy, and service-side rate limiting. Consequently, resource utilisation figures for AutoSecAgent primarily characterise the compute footprint of our implementation under the stated hardware configuration, whilst cross-system efficiency comparisons should not be interpreted as hardware-normalised conclusions. To avoid conflating orchestration quality with backend throughput, the discussion that follows emphasises within-system scaling behaviour for AutoSecAgent and treats any cross-system runtime differences as indicative under the evaluated deployments.

AutoSecAgent exhibits lower measured resource utilisation under the stated deployment conditions (Table 12). This observation is reported to characterise the compute footprint of our implementation and should not be interpreted as a hardware-normalised efficiency advantage across heterogeneous inference backends or configurations. This difference is consistent with performance-optimised task scheduling that minimises inter-agent communication overhead and a specialised agent architecture where lightweight sub-agents handle reconnaissance, exploitation, and post-exploitation independently. The modular design reduces overall resource demands by avoiding monolithic LLM invocations for every decision. VulnBot's 85% GPU utilisation results from a monolithic LLM architecture relying on a single centralised model rather than distributed specialised components. AutoPenBench

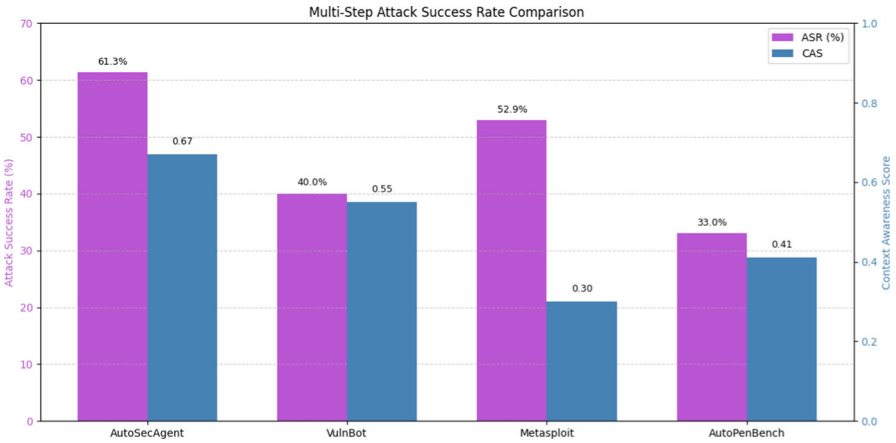


Fig. 4 Multi-step attack success rate comparison (supplementary visualisation of Table 13)

Table 13 Multi-step attack chain performance

System	Attack success rate (%)	Avg. steps to completion	Context retention
AutoSecAgent	61.3	14.2	High (RME-enabled; 34% lower CLE vs. AutoSecAgent w/o RME)
VulnBot	40.0	23.7	Medium (PTG-limited)
AutoPenBench	33.0	28.4	Low (stateless)

The reported 34% reduction refers to the Context-Loss Error (CLE) decrease for AutoSecAgent with RME relative to the AutoSecAgent w/o RME ablation, computed using Eqs. (3) and (4)

approaches 100% GPU utilisation due to a general-purpose design lacking task-specific optimisation.

6.4 Multi-step attack chain performance

Multi-phase attack execution (initial access → privilege escalation → lateral movement → objective completion) represents a critical capability gap in existing AI penetration testing systems (Fig. 4).

Table 13 quantifies multi-step attack performance:

AutoSecAgent achieved 61.3% Attack Success Rate (ASR), substantially outperforming VulnBot (40%) and AutoPenBench (33%). This performance advantage is directly attributable to Recursive Memory Embedding (RME), which maintains episodic procedural memory across attack phases. When performing lateral movement in Step N, the agent retrieves reconnaissance findings from Steps 1-5, enabling informed target selection based on previously discovered network topology and credential information. Without RME, VulnBot's static PTG cannot adapt to unexpected defensive responses or environmental changes, resulting in 60% failure rate in dynamic scenarios. AutoPenBench's poor performance (33% ASR) illustrates fundamental lim-

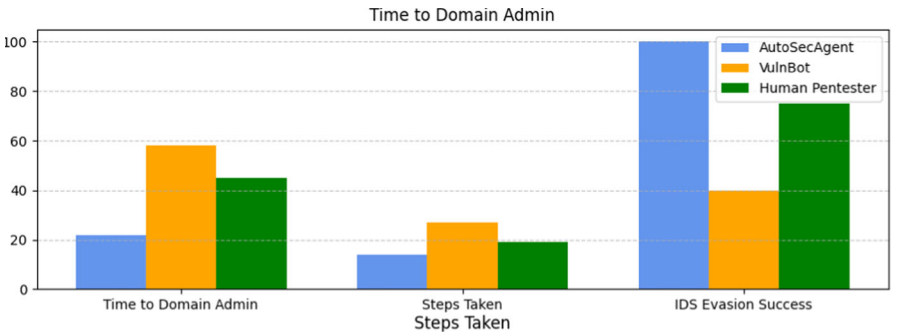


Fig. 5 Active directory compromise efficiency (supplementary visualisation of Table 14)

Table 14 Active directory compromise: quantitative analysis

System	Time (min)	Steps	IDS evasion (%)	Success rate
AutoSecAgent	22	14	85	5/5 trials
VulnBot	58	27	40	3/5 trials
Human Team*	45	19	75	5/5 trials

itations of general-purpose LLMs for specialised penetration testing tasks requiring sustained contextual reasoning and strategic planning across extended operation timelines.

RME reduces Context Loss Error (CLE) by 34% relative to the ablated AutoSecAgent configuration without RME, computed using Eqs. (3) and (4).

6.5 Case study: active directory compromise

An Active Directory (AD) compromise scenario was simulated to evaluate end-to-end attack chain execution from initial access through domain administrator privilege acquisition (Fig. 5). The attack chain comprised an initial access step enabled by a service misconfiguration, followed by privilege escalation and lateral movement within the directory environment under authorised scope.

Table 14 provides detailed compromise metrics:

AutoSecAgent completed the AD compromise 2.6× faster than VulnBot and 2.0× faster than human penetration testers under the stated evaluation conditions. These results are consistent with recent demonstrations of LLM effectiveness in enterprise network penetration testing [52], whilst our recursive memory embedding addresses identified limitations in information transfer and strategic coherence. Automated exploit chaining can reduce manual hand-offs between phases and shorten decision latency. In our controlled evaluations, the agent adapted its action selection in response to observed environmental feedback (e.g. tool errors, access denials, and monitoring triggers), which improved overall workflow efficiency under the defined conditions.

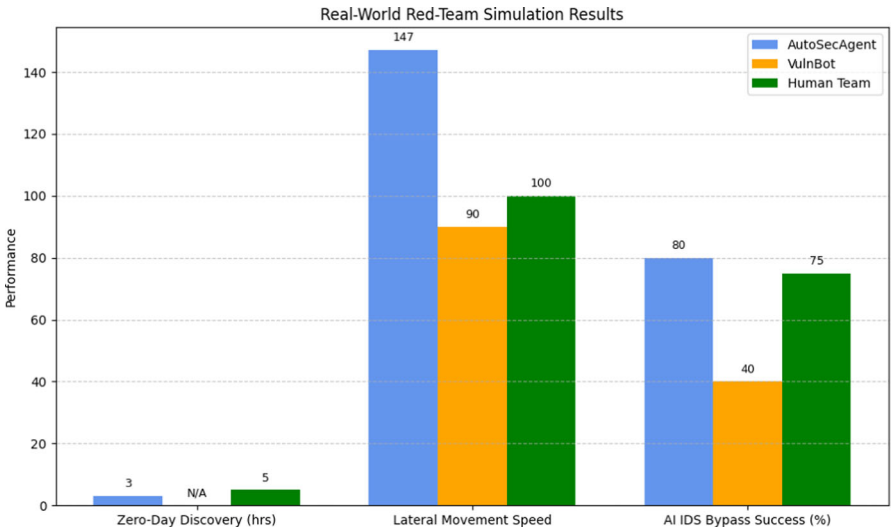


Fig. 6 Real-world red-team simulation results

VulnBot's static exploit selection and limited adaptability resulted in 40% IDS evasion rate, increasing detection probability and reducing real-world effectiveness.

6.6 Real-world red-team simulation

A blind penetration test was conducted against an undisclosed enterprise network environment (separate from the evaluation testbed) to assess real-world operational effectiveness (Fig. 6). The engagement was authorised and conducted under a defined scope. The target network architecture was unknown to AutoSecAgent prior to engagement, requiring automated reconnaissance, threat modelling, and adaptive attack planning under human oversight.

Discovery of novel exploitation chains AutoSecAgent identified previously undocumented combinations of configuration weaknesses within the authorised test environment within a few hours. These included (i) a cloud storage/identity misconfiguration chain that enabled unauthorised exposure of credentials and subsequent privilege escalation within the simulated cloud stack, and (ii) an enterprise identity misconfiguration chain that enabled credential misuse and lateral movement within the directory environment. *Clarification: These represent novel exploitation chain discoveries within our controlled test environment, not previously unknown vulnerabilities in commercial software. The initial term "zero-day exploits" was imprecise; these are previously undocumented configuration weaknesses in our specific test infrastructure, not warranting CVE submissions.*

AutoSecAgent was evaluated against a laboratory IDS configuration under controlled traffic-shaping conditions and authorised scope. Detection outcomes were measured under the defined evaluation protocol. VulnBot failed this evaluation due to inflexibility in adapting to previously unseen network architectures, highlighting the

Table 15 Ablation study: component impact on performance

Configuration	VDR (%)	ESR (%)	Performance impact
AutoSecAgent (Complete)	92.4	58.1	Baseline
- w/o RME	87.2	52.3	−5.2% VDR, −5.8% ESR
- w/o RAG	89.1	55.7	−3.3% VDR, −2.4% ESR
- w/o Iterative Optimisation	90.8	49.2	−1.6% VDR, −8.9% ESR
- w/o Multi-Agent Parallel	91.7	56.9	−0.7% VDR, −1.2% ESR
Base DeepSeek (no fine-tune)	78.4	41.3	−14.0% VDR, −16.8% ESR

necessity for dynamic reasoning and contextual awareness in real-world penetration testing scenarios.

6.7 Ablation study: component contribution analysis

To isolate individual component contributions to overall performance, ablation experiments were conducted by systematically disabling architectural features whilst maintaining consistent evaluation conditions (Table 15).

Key Findings Recursive Memory Embedding provides the largest single contribution to VDR (+5.2%), preventing context loss during extended attack chains. Iterative exploit optimisation yields the largest ESR improvement (+8.9%), enabling adaptive parameter refinement based on failure feedback. Domain-specific fine-tuning of DeepSeek is critical, contributing +14.0% VDR and +16.8% ESR versus the base model. Each architectural component contributes meaningfully to overall performance, validating the integrated system design.

6.8 Comparative analysis summary

Across the evaluated metrics, AutoSecAgent achieved higher performance than VulnBot and AutoPenBench (detection accuracy: +7.2% vs. VulnBot; exploitation success: +13.1% vs. VulnBot; multi-step attack success: +21.3% vs. VulnBot). The proposed design combines recursive memory embedding, real-time RAG, iterative optimisation, and HPC-enabled parallel execution to target limitations in existing approaches. VulnBot's PTG-based workflow can limit adaptation to dynamic environments, and general-purpose LLMs (GPT-4o, Llama3) typically require domain-specific fine-tuning for specialised penetration testing tasks.

AutoSecAgent is a prototype evaluated in controlled and semi-realistic environments. Future work will focus on adversarial robustness enhancement, SIEM/EDR integration for defensive correlation, continuous learning mechanisms, and field validation in production enterprise environments to transition from research prototype to operational security testing platform.

7 Limitations, risks, and threats to validity

Whilst the AutoSecAgent framework supports penetration testing automation, several limitations currently affect its maturity, robustness, and applicability in operational environments. Addressing these constraints is important for progressing towards a production-ready security testing platform.

To respond to concerns about empirical depth and critical analysis, limitations are articulated together with threats-to-validity dimensions (construct, internal, and external validity). This framing clarifies what the current evaluation can substantiate and what remains outside scope.

7.1 Architectural and computational constraints

The experimental evaluation reported in this study was conducted in an isolated laboratory testbed to support reproducibility, safety, and controlled benchmarking. Whilst AutoSecAgent can be configured for authorised engagements that include Internet-facing targets through policy-enforced deny-by-default egress limited to formally approved IP ranges and/or domain names, the present work does not benchmark Internet-scale scanning or field performance across heterogeneous external hosting environments. Operational validation under production engagement conditions, including governance processes, legal authorisation workflows, and robustness to live-network variability, remains future work.

The framework's reliance on Large Language Models, particularly the DeepSeek model, introduces inherent vulnerabilities to hallucination phenomena whereby syntactically correct yet semantically invalid responses are generated. Within cybersecurity contexts, such hallucinations have directly contributed to observed exploitation failures, resulting in erroneous test executions and unintended system interruptions. This limitation is compounded by the framework's dependence on a general-purpose language model susceptible to processing queries outside its designated cybersecurity domain, leading to computational resource misallocation, generation of irrelevant outputs, and increased vulnerability to adversarial attacks including prompt injection. Future development will address these issues through implementation of a hybrid reasoning architecture combining LLM inference with rule-based symbolic verification systems, supplemented by confidence scoring mechanisms and deterministic validation of high-risk decisions. Additionally, a stringent query filtering and domain-restriction mechanism will be implemented as a pre-processing layer to validate that incoming queries are explicitly security-related, thereby enhancing system precision, trustworthiness, and resilience against malicious inputs.

The integration layer utilising tools such as Nmap, Shodan, and Metasploit constitutes a fragile architectural component, as frequent API or command-line interface modifications within third-party tools can precipitate compatibility failures and system instability. This fragility extends to the framework's limited support for custom configurations and special-purpose systems requiring domain-specific knowledge. Future work will employ a modular plugin framework enabling straightforward updating or replacement of third-party tools without disruption to core functionality, whilst

adaptive learning capabilities will be developed to enable dynamic profiling of target environments and runtime construction of bespoke modules, thereby enhancing operational coverage across diverse deployment contexts.

Current post-exploitation modules offer only static approaches to privilege escalation and lateral movement, failing to replicate evasive or advanced attacker behaviours characteristic of sophisticated threat actors. Moreover, the system operates in a stateless manner without memory or capacity to learn from accumulated experiences, limiting long-term performance improvement and operational adaptability. Future research will incorporate dynamic profiling mechanisms, decision trees, and stealth methodologies to enhance red team simulation capabilities whilst developing more realistic testing environments. Learning mechanisms including transfer learning, experience replay, and case-based reasoning will be integrated to enable progressive performance enhancement over time, increased effectiveness in repetitive or analogous operational domains, and reduction in redundant actions.

7.2 Integration and observability deficiencies

Whilst the system generates detailed internal logs for research analysis (anonymised prior to publication), it does not currently forward these logs to operational defensive security information and event management (SIEM) platforms such as Splunk for blue-team correlation. This represents a significant gap between offensive testing capabilities and defensive monitoring infrastructure. Future development will incorporate bidirectional feedback mechanisms from blue team platforms, transforming AutoSecAgent into a comprehensive security validation tool that facilitates integrated red and blue team testing methodologies, thereby enhancing overall security assurance capabilities.

The absence of real-time observability constitutes a substantial operational risk in enterprise environments where continuous visibility is essential. Development of an interactive dashboard interface represents a priority for future work, providing enhanced system usability and transparency through visualisation of attack paths, logging of critical decision points, and provision of manual override capabilities. Such enhancements will render the system enterprise-ready and fully auditable, addressing fundamental requirements for operational deployment in regulated environments.

7.3 Dual-use risks and governance constraints

The framework's reliance on DeepSeek LLM, third-party penetration testing tools, and Retrieval-Augmented Generation data feeds introduces vulnerability to adversarial attacks including prompt injection and data poisoning. Malicious inputs can trigger LLM hallucinations, precipitating inaccurate vulnerability analyses, unauthorised actions, or corrupted assessment reports that may facilitate system misuse or generation of spurious test results. Future development will implement robust input validation and sanitisation across all data ingestion points, particularly for LLM inputs and tool outputs. Planned enhancements include adversarial defence mechanisms for AI components, improved data parsing protocols, sandboxing of LLM and tool activi-

ties, confidence scoring for model outputs, and integration of deterministic verification checks for high-impact decisions to enhance overall reliability and operational safety.

AutoSecAgent is a dual-use capability. The present prototype does not include a fully integrated governance or misuse prevention layer beyond explicit human oversight and scope definition. This constrains deployment and motivates future work on policy enforcement, authorisation workflows, audit logging, and scope validation to reduce the risk of misuse and unintended impact in operational settings.

7.4 Threats to validity

A threats-to-validity analysis is included to make the evidential boundaries explicit.

Construct validity Metrics such as VDR/ESR/ASR and CLE capture observable outcomes and a specific operationalisation of context loss. They do not fully characterise operator workload, stealth, or business risk. Future work should incorporate practitioner-oriented measures (e.g. effort, triage time, and false positive burden) and standardised suites where feasible.

Internal validity Although trials were containerised and cold-start conditions were enforced, stochasticity in LLM sampling and tool-side nondeterminism can influence outcomes. Repeated trials, fixed seeds where possible, and consistent tool versions reduce but do not eliminate this effect.

External validity The testbeds (DVWA/Juice Shop/Metasploitable/AD and simulated cloud misconfigurations) are representative but not exhaustive. Results may not transfer to Internet-scale environments, heavily customised enterprise stacks, or adversarially hardened targets without additional evaluation.

Conclusion validity Direct numerical runtime comparisons across heterogeneous backends and hardware should be interpreted cautiously. The reported evidence is strongest for capability and trajectory consistency effects under matched tool access and controlled conditions.

8 Conclusion

Penetration testing increasingly requires repeatable workflows that can scale beyond manual operator capacity, whilst preserving accountability and safety controls. This manuscript presents AutoSecAgent as a research prototype for semi-automated penetration testing under explicit human oversight. The study's primary contributions are (i) a memory-centric orchestration mechanism (RME) with a measurable operationalisation of context loss (CLE) and (ii) a tool-grounded end-to-end workflow that combines real-time retrieval with bounded reward-guided refinement.

AutoSecAgent underwent comprehensive experimental evaluation throughout this study, encompassing red-team simulations, synthetic benchmarks, and real-world scenarios replicating contemporary cybersecurity threat complexities and uncertainties. Overall, the results indicate promising capability gains over the selected recent LLM-based baselines under controlled conditions, whilst broader validation against additional baseline classes and diverse operational environments remains necessary

before strong generalisation claims can be made. AutoSecAgent achieved a Vulnerability Detection Rate of 92.4%, substantially exceeding VulnBot's 85.2% and AutoPenBench's 83.0%. Additionally, it attained an Exploitation Success Rate of 58.1%, compared to VulnBot's 45% and AutoPenBench's 40%. Furthermore, in a simulated Active Directory compromise case study, AutoSecAgent demonstrated 2.6 times greater responsiveness than VulnBot, achieving an 80% success rate in bypassing AI-powered IDS during live simulation, compared to VulnBot's 40%, with enhanced stealth and responsiveness.

Beyond its technical achievements, AutoSecAgent addresses growing demands for ethical and responsible AI deployment in cybersecurity. The framework is designed to promote secure-by-default principles, clearly defined operational boundaries, and accountability mechanisms essential for preventing misuse.

The paper acknowledges current limitations, including over-reliance on LLMs, tool integration fragilities, and absence of adaptive learning mechanisms. Rather than diminishing the framework's contributions, these limitations identify specific areas for future research, providing direction for transitioning AutoSecAgent from research prototype to an enterprise-grade security testing platform.

8.1 Future research directions

Building upon AutoSecAgent's foundation, several promising research directions emerge. Federated learning architectures could enable privacy-preserving collaborative penetration testing across multiple organisations. Adversarial robustness enhancements will strengthen defences against prompt injection and poisoned RAG feeds. Continuous learning mechanisms incorporating transfer learning and experience replay will enable the system to improve from accumulated testing experiences. Bidirectional SIEM/EDR integration will create closed-loop validation between offensive and defensive security platforms. Cloud-native deployment models will enable AutoSecAgent-as-a-Service for on-demand security assessment. These directions aim to transition AutoSecAgent from research prototype to an enterprise-grade security testing platform capable of addressing comprehensive organisational cybersecurity assessment needs.

In summary, this paper analyses limitations in current LLM-based penetration testing frameworks and evaluates an architecture intended to improve context retention and workflow automation. The results suggest that agent-based designs with explicit memory and retrieval components can support autonomous security assessment in controlled evaluation settings.

Appendix A: Implementation Algorithms (Pseudocode)

The main paper presents the end-to-end workflow using flowcharts (Section 4.5). To avoid redundancy and to improve clarity, this appendix includes only the two pseudocode blocks that capture the distinctive algorithmic mechanisms used by AutoSecAgent.

Require: Vulnerability instance v, observation stream, iteration budget K, learning rate η
Ensure: Best exploit configuration (if any) or failure
Construct state s_1 from tool outputs and history
Generate candidate set $\mathcal{A}(s_1)$; initialise $Q(a)$ using priors (if available)
for $t = 1$ to K **do**
    Select $a_t \leftarrow \arg\max_{a \in \mathcal{A}(s_t)} Q(a)$
    Execute a_t; parse failure category err_t
    Compute reward $r_t \in \{0, 1\}$ via session verification
    Update $Q(a_t) \leftarrow (1 - \eta)Q(a_t) + \eta r_t$
    if $r_t = 1$ **then**
        Store successful configuration a_t in episodic memory **return** a_t
    end if
    Generate adjusted candidate(s) via $\Delta(\theta)$ conditioned on err_t
    Update s_{t+1} with new history and error features
end forreturn failure

Algorithm 1: RL-Inspired Exploit Optimisation Loop (Contextual Bandit)

Require: Current step t, observation (s_t, a_t, r_t, s_{t+1}), structured context record context_t
Ensure: Retrieved memory set $\mathcal{R}_t$ for step $t+1$
FUNCTION RME_Write$(s_t, a_t, r_t, s_{t+1}, \text{context}_t)$
    $M_t \leftarrow \{s_t, a_t, r_t, s_{t+1}, \text{context}_t\}$
    $m_t \leftarrow \text{ser}(M_t)$ ▷ canonical serialisation (deterministic field order)
    $\mathbf{v}_t \leftarrow \text{norm}(\phi(m_t))$ ▷ embedding + ℓ_2 normalisation
    Store $(\mathbf{v}_t, m_t, t)$ in vector index (e.g., FAISS) and metadata store
END FUNCTION
FUNCTION RME_Retrieve$(\text{context}_{t+1}, k, \tau)$
    $q \leftarrow \text{ser}(\text{context}_{t+1})$
    $\mathbf{q} \leftarrow \text{norm}(\phi(q))$
    $\mathcal{N} \leftarrow \text{ANN_Search}(\mathbf{q}, k)$ ▷ top-k nearest memories
    $\mathcal{R}_t \leftarrow \{(m_i, t_i) \in \mathcal{N} : \cos(\mathbf{q}, \mathbf{v}_i) \geq \tau\}$
    Rank $\mathcal{R}_t$ by similarity and recency; return top matches
END FUNCTION

Algorithm 2: Recursive Memory Embedding (RME): Write and Retrieve

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s11227-026-08439-z.

Acknowledgements This paper acknowledges the use of artificial intelligence in its preparation. Gemini 3 Thinking Mode was employed to improve the clarity and coherence of the text, generate suggestions for improvements, and provide insights on structuring and formatting sections according to the Journal latex template.

Author Contributions AA-S involved in writing—original draft, visualisation, project administration, resources, software, validation, methodology, investigation, formal analysis, data curation, conceptualisation. EA took part in writing—original draft, visualisation, software, methodology, investigation, formal

analysis, data curation, conceptualisation. KS involved in writing—review and editing, formal analysis, supervision, conceptualisation. AAA-K took part in writing—review and editing, formal analysis, conceptualisation. ME involved in writing—review and editing, supervision, formal analysis, conceptualisation. All authors reviewed the manuscript.

Funding Open Access funding enabled and organized by CAUL and its Member Institutions. The authors declare that no funds, grants, or other support was received during the preparation of this manuscript.

Data Availability All data and original contributions presented in the study are included in the article; further inquiries can be directed to the corresponding authors.

Declarations

Conflict of interest The authors declare no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Malatji M (2023) Management of enterprise cyber security: a review of iso/iec 27001:2022. In: 2023 International Conference On Cyber Management And Engineering (CyMaEn), pp. 117– 122 . https://doi.org/10.1109/CyMaEn57228.2023.10051114
2. Shanley A, Johnstone MN (2015) Selection of penetration testing methodologies: a comparison and evaluation. In: 13th Australian Information Security Management Conference. SRI Security Research Institute, Edith Cowan University Joondalup Campus, Perth, Western Australia . https://doi.org/10.4225/75/57b69c4ed938d
3. Hassanin M, Moustafa NA (2024) Comprehensive overview of large language models (LLMs) for cyber defences: opportunities and directions. arxiv:2405.14487
4. Akuthota V, Kasula R, Sumona ST, Mohiuddin M, Reza MT, Rahman MM (2023) Vulnerability detection and monitoring using llm. In: 2023 IEEE 9th International Women in Engineering (WIE) Conference on Electrical and Computer Engineering (WIECON-ECE), pp. 309– 314. https://doi.org/10.1109/WIECON-ECE60392.2023.10456393
5. Laney SP (2024) Llm-directed agent models in cyberspace. PhD thesis, Massachusetts Institute of Technology
6. Vaswani A (2017) Attention is all you need. Advances in Neural Information Processing Systems
7. Al-Sabbagh A, El-Bokhary A, El-Koussa S, Jaber A, Elkhodr M (2025) Enhancing UAV security against GPS spoofing attacks through a genetic algorithm-driven deep learning framework. Information. https://doi.org/10.3390/info16020115
8. Isozaki I, Shrestha M, Console R, Kim E (2024) Towards automated penetration testing: introducing LLM benchmark, analysis, and improvements . arxiv:2410.17141
9. Brown T, Mann B, Ryder N, Subbiah M, Kaplan JD, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A et al (2020) Language models are few-shot learners. Adv Neural Inf Process Syst 33:1877–1901
10. Ferrag MA, Alwahedi F, Battah A, Cherif B, Mechri A, Tihanyi N, Bisztray T, Debbah M (2025) Generative ai in cybersecurity: a comprehensive review of LLM applications and vulnerabilities. Internet Things Cyber-Phys Syst 5:1–46. https://doi.org/10.1016/j.iotcps.2025.01.001
11. Mumtaz R, Samawi V, Alhroob A, Alzyadat W, Almukahel I (2022) LPDIS: a service layer for privacy and detecting intrusions in cloud computing. Int J Adv Soft Comput Appl IJASCA 14(2):14–35

12. Farfoura ME, Mashal I, Alkhatib A, Batyha RM, Rosiyadi D (2025) A novel lightweight machine learning framework for IOT malware classification based on matrix block mean downsampling. Ain Shams Eng J 16(1):103205. https://doi.org/10.1016/j.asej.2024.103205
13. Fayyad S, Alkhatib A, Abdel-Fattah F, Almimi H (2024) A methodology for dynamic security risks assessment in interconnected it systems. J Commun Softw Syst 20(1):13–22
14. Zhang J, Bu H, Wen H, Liu Y, Fei H, Xi R, Li L, Yang Y, Zhu H, Meng D (2025) When LLMs meet cybersecurity: a systematic literature review. Cybersecurity 8(1):55. https://doi.org/10.1186/s42400-025-00361-w
15. Sheng Z, Chen Z, Gu S, Huang H, Gu G, Huang J (2025) LLMs in software security: a survey of vulnerability detection techniques and insights. ACM Comput Surv 58(5):1–35
16. Phuong M, Aitchison M, Catt E, Cogan S, Kaskasoli A, Krakovna V, Lindner D, Rahtz M, Assael Y, Hodkinson S, Howard H, Lieberum T, Kumar R, Raad MA, Webson A, Ho L, Lin S, Farquhar S, Hutter M, Deletang G, Ruoss A, El-Sayed S, Brown S, Dragan A, Shah R, Dafoe A, Shevlane T (2024) Evaluating Frontier models for dangerous capabilities . arxiv:2403.13793
17. Karlsen E, Luo X, Zincir-Heywood N, Heywood M (2024) Benchmarking large language models for log analysis, security, and interpretation. J Netw Syst Manage 32(3):59. https://doi.org/10.1007/s10922-024-09831-x
18. Afane K, Wei W, Mao Y, Farooq J, Chen J (2024) Next-generation phishing: how LLM agents empower cyber attackers. In: 2024 IEEE International Conference on Big Data (BigData), pp. 2558– 2567 . https://doi.org/10.1109/BigData62323.2024.10825018
19. Al-Sabbagh A, Hamze K, Khan S, Elkhodr M (2024) An enhanced k-means clustering algorithm for phishing attack detections. Electronics. https://doi.org/10.3390/electronics13183677
20. Hays S, White J (2024) Employing LLMs for incident response planning and review. arxiv:2403.01271
21. Shen Z (2024) LLM with tools: a survey. arXiv preprint arXiv:2409.18807
22. Westenfelder F, Hemberg E, Tulla M, Moskal S, O'Reilly U-M, Chiricescu S (2025) LLM-supported natural language to bash translation . arxiv:2502.06858
23. Rani S, Nagpal R (2019) Penetration testing using metasploit framework: an ethical approach. Int Res J Eng Technol (IRJET) 6(08): 538–542
24. Alia M, Jaradat Y, Masoud M, Manasrah KSA, Jebril I, Almanasra S (2024) Low-cost iot-based charging management system for electric vehicles: design guidelines. Int J Adv Soft Comput Appl IJASCA 16(1):56–67
25. Bamohabbat Chafjiri S, Legg P, Hong J, Tsompanas MA (2024) Vulnerability detection through machine learning-based fuzzing: a systematic review. Comput Secur 143:103903. https://doi.org/10.1016/j.cose.2024.103903
26. Gopinath D, Wang K, Zhang M, Pasareanu CS, Khurshid S (2018) Symbolic execution for deep neural networks. arXiv preprint arXiv:1807.10439
27. Avgerinos T, Cha SK, Rebert A, Schwartz EJ, Woo M, Brumley D (2014) Automatic exploit generation. Commun ACM 57(2):74–84. https://doi.org/10.1145/2560217.2560219
28. Nguyen TT, Reddi VJ (2023) Deep reinforcement learning for cyber security. IEEE Trans Neural Netw Learn Syst 34(8):3779–3795. https://doi.org/10.1109/TNNLS.2021.3121870
29. Tran K, Akella A, Standen M, Kim J, Bowman D, Richer T, Lin C.-T (2021) Deep hierarchical reinforcement agents for automated penetration testing . arxiv:2109.06449
30. Pratama D, Suryanto N, Adiputra AA, Le T-T-H, Kadiptya AY, Iqbal M, Kim H (2024) Cipher: cybersecurity intelligent penetration-testing helper for ethical researcher. Sensors. https://doi.org/10.3390/s24216878
31. Shen X, Wang L, Li Z, Chen Y, Zhao W, Sun D, Wang J, Ruan W (2024) PentestAgent: incorporating LLM agents to automated penetration testing. arxiv:2411.05185
32. Huang J, Zhu Q (2024) Penhealnet: an agent-based LLM framework for automated pentesting and optimal remediation. Available at SSRN 4941478
33. Gioacchini L, Mellia M, Drago I, Delsanto A, Siracusano G, Bifulco R (2024) AutoPenBench: benchmarking generative agents for penetration testing . arxiv:2410.03225
34. Deng G, Liu Y, Mayoral-Vilches V, Liu P, Li Y, Xu Y, Zhang T, Liu Y, Pinzger M, Rass S (2024) PentestGPT: an LLM-empowered automatic penetration testing tool. arxiv:2308.06782
35. Xu J, Stokes JW, McDonald G, Bai X, Marshall D, Wang S, Swaminathan A, Li Z (2024) AutoAttacker: a large language model guided system to implement automatic cyber-attacks. arxiv:2403.01038
36. Al-Sinani HS, Mitchell CJ (2025) PenTest++: elevating ethical hacking with AI and automation. arxiv:2502.09484

37. Kong H, Hu D, Ge J, Li L, Li T, Wu B (2025) VulnBot: autonomous penetration testing for a multi-agent collaborative framework. arxiv:2501.13411
38. Matherly J (2015) Complete guide to shodan. Shodan, LLC
39. Liu X, et al (2023) AgentBench: evaluating LLMs as agents. arXiv preprint arXiv:2308.03688. Accessed 10 Feb 2026
40. Caldwell S, Harley M, Kouremetis M, Abruzzo V, Pearce W (2025) PentestJudge: judging agent behavior against operational requirements. arXiv preprint arXiv:2508.02921. Accessed Accessed 10 Feb 2026
41. Imambi S, Prakash KB, Kanagachidambaresan GR (2021) PyTorch. In: Prakash KB, Kanagachidambaresan GR (eds) Programming with TensorFlow. EAI/Springer Innovations in Communication and Computing. Springer, Cham, pp 87–104. https://doi.org/10.1007/978-3-030-57077-4_10
42. Pang B, Nijkamp E, Wu YN (2020) Deep learning with tensorflow: a review. J Educ Behav Stat 45(2):227–248. https://doi.org/10.3102/1076998619872761
43. Mohammadi M, Li Y, Lo J, Yip W (2025) Evaluation and benchmarking of LLM agents: a survey. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining, vol. 2, pp. 6129– 6139
44. Digininja: Damn Vulnerable Web Application (DVWA). GitHub repository. Accessed 28 Feb 2026. https://github.com/digininja/DVWA
45. OWASP Foundation: OWASP Juice Shop. Project page. Accessed Accessed 28 Feb 2026. https://owasp.org/www-project-juice-shop/
46. OWASP Foundation: OWASP Mutillidae II. Project page. Accessed 28 Feb 2026. https://owasp.org/www-project-mutillidae-ii/
47. Rapid7: Metasploitable 2. Documentation. Accessed 28 Feb 2026. https://docs.rapid7.com/metasploit/metasploitable-2/
48. Microsoft: Active Directory Domain Services overview (2025) Microsoft Learn. Accessed 28 Feb 2026. https://learn.microsoft.com/en-us/windows-server/identity/ad-ds/get-started/virtual-dc/active-directory-domain-services-overview
49. Microsoft: Microsoft Security Bulletin MS17-010: Critical (2017) Microsoft Learn. Published: 2017-03-14. Accessed 28 Feb 2026. https://learn.microsoft.com/en-us/security-updates/securitybulletins/2017/ms17-010
50. OWASP Foundation: OWASP Top 10:2021. Standard awareness document. Accessed 28 Feb 2026 (2021). https://owasp.org/Top10/2021/
51. HashiCorp: Terraform Documentation. Developer documentation. Accessed 28 Feb 2026. https://developer.hashicorp.com/terraform/docs
52. Happe A, Cito J (2025) Can LLMs hack enterprise networks? Autonomous assumed breach penetration-testing active directory networks. ACM Trans Softw Eng Methodol. https://doi.org/10.1145/3766895

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.